Homework 4

CS 521: Fall 2026

Due on Tuesday, November 10, 2026 11:59 PM Central

Homework Policy: You are allowed to collaborate with your classmates, but report this in your submission. Even if you work on problems together, each student must write up their solution individually in their own words in their submission. This CS 473 course page on academic integrity is a handy reference: `https://courses.engr.illinois.edu/cs473/sp2023/integrity.html`.
We have ZERO tolerance for breaches of academic integrity, such as, but not restricted to, plagiarism, use of LLMs except for when stated explicitly, etc.
You can take upto 3 days extra in total across the semester without explanation. Once you exhaust the three-day late submission allowance, any further late homework submissions will result in a 25% penalty per each extra day. If the submission is late by 5 minutes or more then the submission will be counted as 1 day late. If you have exceptional personal circumstances that will prevent you from submitting the assignment on time, write to the course staff as soon as possible. Unless explicitly stated, use of LLMs for solving homework or cheating from others is not allowed – we will detect it! There will be an oral **exam** if we detect violations, during which you will be asked to answer a randomly selected question from the homework. If you are unable to show sufficient knowledge, then you will get zero for the full homework.
Please format your submission as a PDF (file naming convention: *CS521_⟨netid⟩_hw4.pdf*) and upload it on Gradescope by the deadline. Try to start every problem on a new page, to accurately map pages to questions on Gradescope.
Programming Problems: Describe your solution at a high-level (if applicable). Write only small snippets of code, if at all. DO NOT paste tall walls of code into your submission! Upload your solution files to GitHub publicly and provide a link to the relevant files as part of your solution.

Points: Problem 1: 15 + 10 = 25 points, Problem 2: 10 + 10 + 25 = 45 points, Problem 3: 10 + 20 = 30 points. Total: 100 points.
There are two bonus questions, each worth 25 points. You can solve one or both of them. You get at most 25 points for the bonus section.

Problem 1 (Interval Bound Propagation, Programming)**.** In this problem, you implement interval bound propagation (IBP) training [GDS+19] for a small neural network.

Network. Use a fully connected network with 3 hidden layers. Each hidden layer has 50 neurons and is a linear layer followed by a ReLU. Train the network on MNIST with the cross-entropy loss. You can use the data loaders from HW2: `https://github.com/nalinwadhwa02/CCS521FA26HW/blob/main/hw2/q4/mnist_interval_analysis.ipynb`.

(a) (15 points) Implement IBP training for your network. Use the two training procedures of the IBP paper (see the hints):

- Decrease the weight κ of Equation 12 of the paper gradually from 1 to 0.5.
- Increase the training radius ϵ_{train} gradually from 0 to 0.1.

Report the standard accuracy and the robust accuracy against a PGD attack. Report the training time, and compare it with the time of standard training.

(b) (10 points) Use your box verifier from HW2 to calculate the verified accuracy. The verified accuracy is the percentage of test images for which the verifier proves robustness in the ℓ_∞ ball of radius ϵ_{test}. Use 10 values of ϵ_{test} from 0.01 to 0.1, with equal steps. If the network is not certified for some ϵ_{test}, find adversarial examples in these balls. Show some of these images and analyze them.

Solution requirements. Give your solution as a Jupyter notebook. We recommend Google Colab, because we can then run your solution easily. You can also put the notebook in your GitHub repository.

Hints. IBP adds a *robustness loss* term to the training loss. Let $CE(\cdot, \cdot)$ be the cross-entropy loss, and z_K the logits of the last layer K. The loss is:

$$L_{IBP} = \kappa \cdot CE(z_K, y_{true}) + (1 - \kappa) \cdot CE(\hat{z}_K(\epsilon_{train}), y_{true})$$

Merge the last linear layer into the robustness specification, as in Equation 9 of the paper. Change the loss function of your training loop, and add the two training procedures of the paper. For an example implementation, see `https://github.com/Zinoex/bound_propagation/blob/main/examples/fashion_mnist.py`.

Problem 2 (Certified Training for Multiple Norms, Programming)**.** A network that is certified for one norm is frequently not certified for other norms. CURE [JCS24] is a certified training method for robustness against multiple norms. In this problem, you train networks with a simplified version of CURE, and then you improve the method.

A network is *certified* at an image x for the norm p if the verifier proves that the network classifies all images x' with $\|x' - x\|_p \leq \epsilon_p$ correctly. We use $\epsilon_\infty = 0.1$ and $\epsilon_2 = 1.0$. The *certified accuracy* for a norm is the percentage of test images where the network is certified for that norm. The *union certified accuracy* is the percentage of test images where the network is certified for both norms.

Starter code. The starter code is the Colab notebook `https://github.com/nalinwadhwa02/CCS521FA26HW/blob/main/hw4/q2/multi_norm_certified_training_mnist.ipynb`. Open it in Google Colab and select a GPU runtime (*Runtime > Change runtime type > T4 GPU*). The notebook gives the MNIST data, the network of Problem 1, the ℓ_∞ and ℓ_2 PGD attacks, and a verifier that uses the tighter of the IBP and the CROWN bounds. The function `train(model, loss_fn)` trains a network for 20 epochs, with the same ϵ schedule as in Problem 1. For each training method, you write the loss function `loss_fn`. The function `evaluate` gives the clean accuracy, the certified accuracy and the PGD accuracy for each norm and for the union. Use `evaluate` for all the numbers in your report, and do not change the verifier or `evaluate`.

SABR. CURE uses the SABR loss from the lectures for each norm. SABR finds an adversarial example x^* with PGD, and then it calculates the IBP loss on a small ball of radius $\tau = \lambda\epsilon$ around x^*. The ratio $\lambda \in (0, 1]$ sets the size of the small ball. PGD searches for x^* in the ball of radius $\epsilon - \tau$ around x, so that the small ball is inside the full ball.

(a) (10 points) **Single-norm certified training.** Complete the function `propagation_region`. It returns the center x^* and the radius τ of the small ball. Then complete the function `certified_loss`. It returns the cross-entropy loss of $\hat{z}$ (as in Problem 1) on the small ball. Then complete the function `single_norm_loss`. It returns the standard loss in the first epoch. After that, it returns `certified_loss` with the radius of the ϵ schedule.

Train three networks: one with standard training, one with ℓ_∞ certified training and one with ℓ_2 certified training. Give the results and the training time for each epoch in a table. Answer these questions:

(i) Does certified robustness for one norm transfer to the other norm? Compare with the PGD accuracy.

(ii) For each network, which norm limits the union certified accuracy?

(b) (10 points) **CURE.** CURE trains for the two norms at the same time. CURE-Max uses the larger of the two SABR losses for each image. CURE also adds a *bound alignment* (BA) loss, which makes the ℓ_2 bounds of an image similar to its

ℓ_∞ bounds. Let $\hat{z}_i^p$ be the $\hat{z}$ of image i on its small ball for the norm p, and let $d_i^p = \text{softmax}(\hat{z}_i^p)$. The CURE loss is

$$\mathcal{L}_{CURE} = \frac{1}{n}\sum_{i=1}^{n} \max\left(\mathcal{L}_{CE}(\hat{z}_i^\infty, y_i),\ \mathcal{L}_{CE}(\hat{z}_i^2, y_i)\right) + \eta \cdot \frac{1}{|\gamma|}\sum_{i\in\gamma} \text{KL}\left(d_i^2 \,\|\, d_i^\infty\right),$$

where γ is the set of images in the batch that are certified for ℓ_2 on their small ball. Complete the function `cure_loss`.

CURE also uses the gradient projection (GP) of RAMP. Let g_c be the gradient of $\mathcal{L}_{CURE}$, and g_n the gradient of the cross-entropy loss on the clean images. For each parameter tensor l, the update direction is $g^l = \beta \cdot \text{GP}(g_n^l, g_c^l) + (1-\beta)\cdot g_c^l$. Complete the function `gradient_projection`. The function `train` uses it if you give the argument `gp_beta`.

Train three networks: CURE-Max ($\eta = 0$, without GP), CURE-Max + BA ($\eta = 0.5$, without GP) and CURE-Max + BA + GP ($\eta = 0.5$, $\beta = 0.5$). Train each network with two seeds (`seed=0` and `seed=1`), and give the mean of the two runs. Give the results in a table together with the results from part (a). Answer these questions:

(i) Compare CURE with the single-norm networks. Which network gives the highest union certified accuracy? Which network gives the highest clean accuracy?

(ii) Do BA and GP increase the union certified accuracy in your runs? Tell why or why not.

(iii) How much more time does CURE use than single-norm training? Why?

(c) (25 points) **Improve CURE.** Change the training method to get a higher union certified accuracy, or a higher clean accuracy with the same union certified accuracy. Your method must be different from the methods of part (b). A change of hyperparameters only (for example η, β, λ, the learning rate or the number of epochs) is not a new method. Use the same network, and do not change the verifier or `evaluate`. You can get ideas from other papers, but you must cite them and tell what is new in your method. Do these steps:

(i) **Idea.** Describe your method. Give the training objective or the pseudocode. Tell why you expect that it increases the union certified accuracy.

(ii) **Implementation.** Write the loss of your method in the function `my_loss` in the notebook, and train with `train` with the default settings, as for the

other methods. If your method needs a different update, write your own copy of `train`.

(iii) **Results.** Compare your method with the best network from part (b) in one table. Run your method with two seeds and give the mean, and give the training time for each epoch. If your method uses more training time than CURE, also compare it with CURE for the same training time (for example, train CURE for more epochs).

(iv) **Ablation.** Remove or change one part of your method. Show how the results change.

(v) **Analysis.** Does your method work? If it works, tell why. If it does not work, tell why not. Use evidence from your experiments, for example the certified accuracy for each norm, the difference between the certified and the PGD accuracy, or the training loss. A method that does not work can get full points if your analysis is good.

Discuss your process and steps in solving the problem, as well as put your notebook on GitHub and give the link. We grade part (c) as follows: idea and justification (5 points), implementation and results (10 points), ablation and analysis (10 points).

Hints.

- Look at the certified accuracy for each norm to find which norm limits the union certified accuracy. A large difference between the certified and the PGD accuracy shows that the bounds are loose.

- Some possible directions: the bounds that the loss uses, how to combine the losses of the two norms, the small ball for each norm, and the images that the loss focuses on.

Problem 3 (Reinforcement Learning: PPO and LLM Post-Training, Programming)**.** In the lectures, we discussed proximal policy optimization (PPO) [SWD$^+$17] and the post-training of LLMs with RL. In part (a), you train a PPO agent on the CartPole environment. In part (b), you compare LLM checkpoints before and after preference tuning and RL.

In CartPole-v1, the agent moves a cart to the left or to the right to keep a pole upright. The agent gets a reward of +1 for each step. An episode ends when the pole falls, or after 500 steps. Thus the maximum return of an episode is 500.

Starter code. The starter code for part (a) and for Bonus Research Question 2 is the Colab notebook `https://github.com/nalinwadhwa02/CCS521FA26HW/blob/main/hw4/q3/ppo_cartpole.ipynb`. The Colab CPU runtime is sufficient. The notebook gives a complete implementation of PPO: the environments, the actor and the critic networks, the advantage calculation with GAE [SML+16], the PPO update with the clipped objective, and the training loop `train_ppo`. Each training run uses 200,000 environment steps. The function `run_seeds` trains one agent for each of 5 seeds, and the function `summarize` gives these metrics:

- `auc`: the mean return during training (the mean of the learning curve). A higher value means faster learning.
- `steps_to_475`: the first step where the mean return of the last 10 episodes is at least 475.
- `final`: the return of the greedy policy at the end of training.

The function `plot_curves` plots the learning curves. Use these functions for all the numbers and plots in your report. Do not change the environment, the number of environment steps or the evaluation.

(a) (10 points) **Run PPO.** Train PPO with the default settings for the 5 seeds. Then train two changed versions: PPO without clipping ({``clip_coef'': 1e9}) and PPO without GAE ({``gae_lambda'': 1.0}). Give the metrics in a table, and plot the learning curves. Answer these questions:

(i) How does the clipping change learning? Why?

(ii) How does GAE change learning? Use the bias and the variance of the advantage estimates to tell why.

(iii) Does PPO keep the maximum return after it gets there? Use the learning curves.

(b) (20 points) **Preference tuning and RL in LLMs.** An LLM is usually trained in several steps after pretraining: supervised fine-tuning (SFT) on instructions, preference tuning (RLHF [OWJ+22] or DPO [RSM+24]), and sometimes RL with verifiable rewards (RLVR [LMP+24]). For OLMo 2 1B [TWS+24], the checkpoint after each step is public on Hugging Face:

- `allenai/OLMo-2-0425-1B`: the base model after pretraining.
- `allenai/OLMo-2-0425-1B-SFT`: the base model after SFT.
- `allenai/OLMo-2-0425-1B-DPO`: the SFT model after DPO.
- `allenai/OLMo-2-0425-1B-Instruct`: the DPO model after RLVR.

The models are small enough for a Colab T4 GPU. Compare the four checkpoints on public benchmarks:

(i) Select at least two public benchmarks that measure different properties. Examples are TruthfulQA (truthfulness), GSM8K (math reasoning), IFEval (instruction following), MMLU (knowledge), and a toxicity or safety benchmark. Evaluate the four checkpoints on the same examples of each benchmark (for example, 200 examples). Use the same prompts and decoding settings for all checkpoints, and use the chat template for the SFT, DPO and Instruct checkpoints. You can use an evaluation library, for example `lm-evaluation-harness`. Give the results in a table, and show some example outputs.

(ii) Use your results to answer these questions. Which properties does SFT change? Which properties does DPO change? Which properties does RLVR change? Does any property get worse after a step (for example, safety, calibration or the diversity of the outputs)? Tell why you think that this occurs.

We grade part (b) as follows: evaluation (12 points), analysis (8 points).

Bonus Research Questions

Bonus Research Question 1 (25 points). You can solve this bonus question, the next bonus question, or both. You get at most 25 points for the bonus section, so the maximum total for the homework is 125 points.

In the lectures, we discussed the first method for certified training against universal adversarial perturbations (UAPs) [XS24]. In this problem, improve this method. The objective is a higher standard accuracy and a higher worst-case UAP accuracy on MNIST. You can use the network of the other problems, or design a new network (for example, an ℓ_∞ UAP network). Give a justification for each part of your procedure. We give up to 15 points for a clear idea with pseudocode, and up to 25 points for the idea, the pseudocode and an implementation.

Bonus Research Question 2 (25 points). **Improve PPO.** Use the PPO notebook of Problem 3. Change the algorithm to make learning faster or more stable. A change of hyperparameters only (for example the learning rate, the clipping coefficient, the number of epochs, the batch size or the size of the networks) is not a new method. Use the same environment, number of environment steps and evaluation. You can get ideas from other papers, but you must cite them. Do these steps:

(i) **Idea.** Describe your change. Tell why you expect that it improves learning.

(ii) **Implementation.** Write your method in the function `my_train_ppo` in the notebook. Start from a copy of `train_ppo`.

(iii) **Results.** Run your method for the 5 seeds. Compare it with PPO in one table, and plot the learning curves.

(iv) **Ablation.** Remove or change one part of your method. Show how the results change.

(v) **Analysis.** Does your change improve learning? If it does, tell why. If it does not, tell why not. Use evidence from your experiments, for example the learning curves, the losses or the size of the gradients. A change that does not improve learning can get full points if your analysis is good.

Put your notebook on GitHub and give the link. We grade this question as follows: idea and justification (5 points), implementation and results (10 points), ablation and analysis (10 points).

Hint. Some possible directions: the scale of the inputs, the rewards or the advantages; what happens at the end of an episode; the updates of the actor and the critic; and the

exploration.

References

[GDS+19] Sven Gowal, Krishnamurthy Dvijotham, Robert Stanforth, Rudy Bunel, Chongli Qin, Jonathan Uesato, Relja Arandjelovic, Timothy Mann, and Pushmeet Kohli. On the effectiveness of interval bound propagation for training verifiably robust models, 2019.

[JCS24] Enyi Jiang, David S Cheung, and Gagandeep Singh. Towards universal certified robustness with multi-norm training. *arXiv preprint arXiv:2410.03000*, 2024.

[LMP+24] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[OWJ+22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

[RSM+24] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024.

[SML+16] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

[SWD+17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[TWS+24] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 OLMo 2 furious. *arXiv preprint arXiv:2501.00656*, 2024.

[XS24] Changming Xu and Gagandeep Singh. Cross-input certified training for universal perturbations, 2024.